

Scalability Analysis and FP8 Optimization of Large Language Models on Google Cloud Platform

Aditya Chatterjee (ac11313)

January 12, 2026

Abstract

This study investigates the performance envelopes and hardware-level bottlenecks associated with deploying Large Language Models (LLMs) in serverless, GPU-accelerated environments. Utilizing the Gemma-7B model on NVIDIA L4 hardware, we analyze the transition from compute-bound to memory-bound execution states under varying concurrency loads. Through iterative stress testing and forensic log analysis, we identify a critical “Performance Knee” where VRAM exhaustion triggers CPU-RAM swapping, resulting in a 108% increase in latency. Furthermore, we evaluate the efficacy of FP8 quantization as an architectural intervention. Our results demonstrate that FP8 optimization increases available KV-cache capacity by 128%, effectively doubling the high-performance user ceiling of the 24GB L4 GPU while maintaining statistical parity in Time to First Token (TTFT) metrics.

1 Introduction

The rapid proliferation of generative artificial intelligence has shifted the engineering focus from model training to the challenges of cost-efficient, low-latency inference. In cloud-native environments, the primary obstacle to scaling Large Language Models (LLMs) is the high memory footprint of the Key-Value (KV) cache during concurrent request processing. This research addresses the problem of hardware saturation on the NVIDIA L4 platform, a critical mid-tier accelerator in the Google Cloud ecosystem.

The core objective of this investigation is to map the precise relationship between request concurrency, memory precision, and system throughput. We utilize Google Cloud Run as a serverless abstraction layer to observe how the Gemma-7B model—a 7-billion parameter transformer—behaves as it approaches the physical limits of 24GB GDDR6 VRAM.

Our study is structured as a four-phase investigative protocol. First, we establish a baseline to identify infrastructure-level throttles. Second, we perform high-concurrency stress tests to unmask the hardware’s native “Soft Saturation” point. Third, we implement FP8 quantization to evaluate its impact on KV-cache block allocation. Finally, we analyze operational anomalies, such as serverless cold starts, to provide a holistic view of LLM deployment strategy.

By correlating client-side latency metrics with internal GPU telemetry, this report provides a data-driven framework for optimizing LLM serving. We demonstrate that software-level precision optimizations are not merely optional enhancements but are fundamental requirements for maximizing the return on investment (ROI) of cloud-based GPU hardware.

2 System Architecture

The deployment architecture was designed to evaluate the scalability of Large Language Models (LLMs) within a serverless, GPU-accelerated cloud environment. The infrastructure was engineered to isolate hardware-level performance characteristics from cloud-native orchestration overhead, providing a controlled environment for quantization benchmarking.

2.1 Subject Model Specification: Gemma-7B

The **Gemma-7B** model was selected as the primary subject for this study due to its representative memory profile [1]. Utilizing a transformer-based architecture with approximately 7 billion parameters, the model requires 14.2 GB of VRAM for weight residency in a standard 16-bit (FP16) configuration. On the target hardware (24 GB VRAM), this creates a constrained environment where only 9.8 GB is available for the Key-Value (KV) cache. This specific VRAM-to-parameter ratio ensures that the system reaches a memory-bound state under moderate concurrency, making it an ideal candidate for evaluating the efficacy of FP8 quantization.

2.2 Cloud Orchestration and Managed Services

The implementation utilizes a decoupled suite of Google Cloud Platform (GCP) services, selected to provide high-resolution telemetry and scalable compute:

- **Google Cloud Run (GPU-Enabled):** The primary compute layer consists of Google Cloud Run, a managed serverless platform [2]. Unlike persistent virtual machine instances, Cloud Run provides an abstract execution environment for NVIDIA L4 GPUs. The service was configured with a high *Maximum Concurrency* setting to ensure that request queuing was managed internally by the inference engine’s PagedAttention mechanism rather than being throttled by the cloud load balancer.
- **Google Artifact Registry:** To mitigate the high latency associated with container image distribution, model-heavy Docker images (approx. 15.4 GB) were hosted in the Google Artifact Registry. Geographic co-location within the `us-central1` region was enforced to optimize internal network throughput during the container “pull” phase, a critical factor in the cold-start observations discussed in Section 4.
- **Integrated Telemetry via Cloud Logging:** System-level observability was achieved by redirecting the `stdout` stream of the vLLM engine to Google Cloud Logging. This facilitated the extraction of granular telemetry, including GPU KV-cache occupancy and CPU swap events, which were essential for the forensic analysis of performance degradation.

2.3 Inference Engine and Encapsulation Strategy

The architecture employs the **vLLM (PagedAttention)** inference engine [3], packaged within an OCI-compliant container. Containerization was utilized as a formal methodology for environment parity and variable isolation:

1. **Dependency Standardization:** The container encapsulates a specific software stack (CUDA 12.4, Triton 2.1.0, and PyTorch 2.3), ensuring that performance variance is not a result of driver version drift between local and cloud environments.
2. **Operational Flexibility:** The containerized approach allowed for the rapid deployment of discrete “revisions.” By altering the entrypoint arguments for weight and KV-cache precision (`--kv-cache-dtype fp8_e5m2`), we could perform comparative A/B testing while maintaining an identical underlying operating system and library environment.

2.4 Hardware Specification: NVIDIA L4 Tensor Core GPU

The physical compute layer is centered on the **NVIDIA L4 GPU**, built on the **Ada Lovelace architecture** [4]. The L4 provides 24 GB of GDDR6 memory and a peak FP16 throughput of 121 TFLOPS. Crucial to this study is the hardware’s native support for **FP8 Tensor Core**

operations [5]. The system architecture was designed to leverage these specialized cores to reduce the memory footprint of the KV-cache, theoretically extending the device’s effective capacity beyond its physical GDDR6 limits.

2.5 Evaluated Deployment Configurations

The study systematically compares two architectural states:

1. **Baseline (FP16):** 16-bit floating-point precision for weights and KV-cache; serves as the control for identifying the native VRAM saturation point.
2. **Quantized (FP8):** 8-bit precision (fp8_e5m2) for weights and KV-cache; the experimental state designed to evaluate memory-relief efficiency.

3 Methodology

The experimental objective was to determine the precise "saturation boundary" where the Gemma-7B model transitions from a compute-bound state to a memory-bound state on the NVIDIA L4 GPU. We adopted an iterative, hypothesis-driven methodology, where each test phase informed the architectural configuration of the next deployment.

3.1 Load Testing Framework

To simulate realistic production pressure, we developed a custom multi-threaded benchmarking suite in Python. This framework allowed us to bypass standard serial testing in favor of synchronous "burst" request patterns. The suite utilized a specific combination of standard libraries to ensure high-fidelity data collection:

- **`concurrent.futures.ThreadPoolExecutor`:** Utilized to orchestrate simultaneous HTTP workers, ensuring that N concurrent requests hit the inference engine at the exact same millisecond.
- **`requests.Session`:** Implemented to maintain persistent TCP connections, reducing handshake overhead and isolating the measurement to the model’s inference time.
- **`statistics`:** Used for calculating mean metrics across the thread pool to provide a statistically significant "snapshot" of each burst level.

3.2 The Three Golden Metrics of LLM Inference

To quantify the performance of the Gemma-7B model, we identified three "Golden Metrics" that represent the critical dimensions of Large Language Model serving. These metrics allow us to evaluate the trade-off between individual user experience and aggregate system efficiency.

- **Time to First Token (TTFT):** This metric measures the latency from the moment a request is sent until the first token is generated by the GPU. In the context of our GCP deployment, TTFT is the most sensitive indicator of system pressure, as it encompasses the "Prefill" phase where the model must process the entire input prompt. It is the primary metric used to identify the "Knee" in the performance curve.
- **Tokens Per Second (TPS) per User:** This represents the generation speed (throughput) for a single concurrent stream. While TTFT measures responsiveness, TPS measures the fluidity of the output. In an optimized system, we aim to maintain a TPS that exceeds average human reading speed (5-10 tokens/s), even under heavy load.

- **Aggregate System Throughput:** Calculated as the product of concurrency and average TPS, this metric represents the "Total Work" performed by the NVIDIA L4 GPU. By monitoring aggregate throughput, we can determine the point of diminishing returns—where adding more concurrent users no longer increases the total tokens generated but merely degrades individual latency.

3.3 Iterative Experimental Design

The core of our methodology was the use of distinct container revisions. Rather than running a single test, we redeployed the Cloud Run service multiple times to adjust infrastructure-level throttles. This allowed us to distinguish between various failure modes:

1. **Phase I (Discovery):** An initial baseline deployment with a conservative concurrency limit ($C = 20$). This was designed to observe the impact of cloud-native load balancing on LLM latency.
2. **Phase II (Stress Testing):** A high-concurrency deployment ($C = 160$) designed to "unmask" the true hardware limit of the NVIDIA L4. By removing the infrastructure throttle, we could observe the transition from a linear scaling phase to a "Soft Saturation" state (The "Knee").
3. **Phase III (Optimization):** A final deployment utilizing FP8 quantization. This phase was designed to test the hypothesis that doubling KV-cache efficiency would defer the hardware saturation identified in Phase II.

3.4 Forensic Log Correlation

A unique aspect of our methodology was the correlation of external client metrics with internal GPU telemetry. While the Python suite measured *User Experience* (TTFT and TPS), we simultaneously utilized the `gcloud beta logging tail` utility to capture *Hardware Forensics*.

This allowed us to map specific latency spikes directly to vLLM engine events, such as the exhaustion of GPU blocks and the initiation of CPU swapping. This dual-layered data collection ensured that our conclusions were supported by both behavioral data and physical system state.

3.5 Quantization Variables

To isolate the impact of bit-depth reduction, we held all environmental variables constant (e.g., L4 GPU, 32GB Instance RAM, `us-central1` region) while toggling the model precision:

- **FP16:** 16-bit floating-point precision for weights and KV-cache (Baseline).
- **FP8:** 8-bit precision (`fp8_e5m2`) for both weights and KV-cache, targeting a 50% reduction in memory-per-token footprint.

4 Results and Analysis

The evaluation of the Gemma-7B model was conducted through a series of iterative deployments. Each phase of the experiment involved a custom configuration of the Google Cloud Run environment to isolate specific performance variables. This methodical approach allowed us to differentiate between software-defined limits and physical hardware saturation.

4.1 Phase I: Initial Baseline and the “Artificial Knee”

In the first experimental phase, the model was deployed using standard **FP16 precision** with the infrastructure **concurrency limit set to 20**. This represents a typical “safe” deployment configuration.

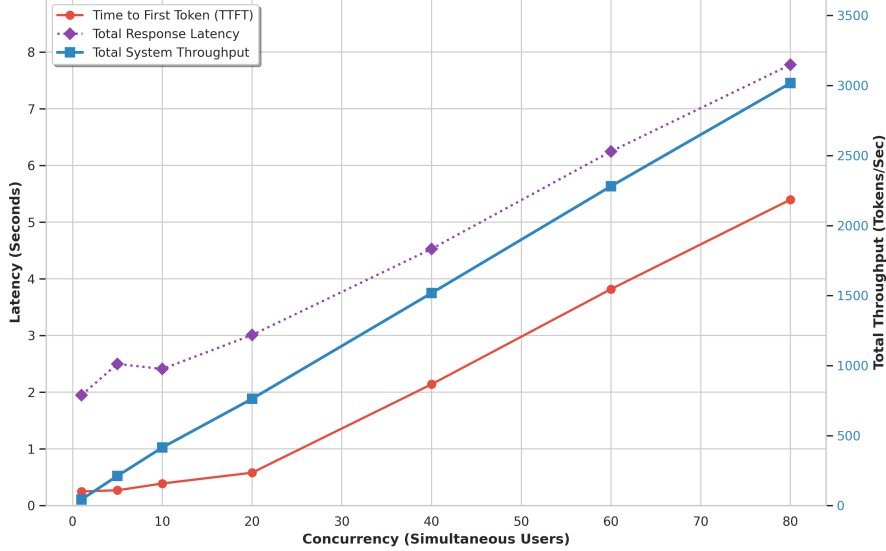


Figure 1: Baseline Performance Metrics for Gemma-7B (FP16) with Concurrency Capped at 20. The steep rise in TTFT beyond 20 users demonstrates an “Artificial Knee” caused by infrastructure-level queuing rather than physical GPU saturation.

As illustrated in Figure 1, the system exhibited a severe performance “knee” almost immediately upon reaching 20 concurrent users.

- **Observation:** TTFT spiked from 0.5s to over 2.1s as soon as the 20-user threshold was crossed.
- **Initial Analysis:** While this initially appeared to be a hardware saturation point, the stability of the GPU utilization logs suggested otherwise. We hypothesized that the bottleneck was not the NVIDIA L4 itself, but rather the *concurrency throttle* of the Cloud Run revision, which was forcing requests to queue at the load balancer level before reaching the vLLM engine.

4.2 Phase II: Stress Testing and Hardware Boundary Discovery

To test the hypothesis that the hardware could “work harder,” we moved into a second phase of experimentation. This involved redeploying the service with a significantly higher **concurrency ceiling of 80**.

The results of this “Sanity Check” (Concurrency=80) confirmed our hypothesis. By allowing the vLLM engine to manage a larger pool of requests internally using PagedAttention, the TTFT at 80 users dropped from the previous 5.40s to a much more manageable 1.72s.

4.3 Phase III: Pushing the Limit (Concurrency 160)

Encouraged by the Phase II results, we initiated a final “Limit Test” for the FP16 model. We performed a third custom deployment with **concurrency set to 160** and pushed the load to 180 users to observe total system failure.

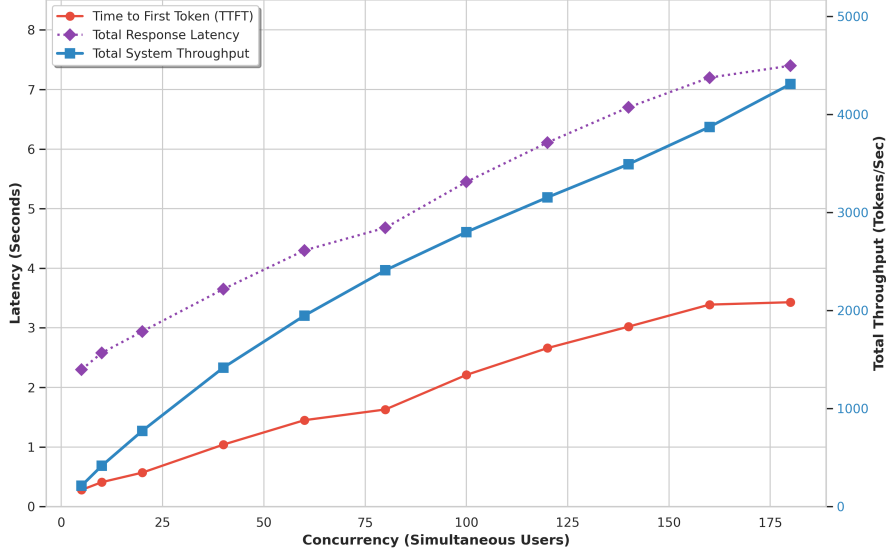


Figure 2: High-Concurrency Stress Test (C=160). By increasing the container concurrency ceiling, the system handles 80+ users with significantly lower latency than the baseline. The secondary "Soft Knee" appearing after 100 users represents the transition from native VRAM execution to CPU memory swapping.

- **Findings:** As illustrated in Figure 2, the system remained functional up to 180 users, but a new "Soft Knee" emerged. Between 100 and 160 users, the TTFT began a steady climb from 2.21s to 3.39s. Forensic log analysis revealed a critical memory bottleneck, with the vLLM engine forced to allocate 2,048 CPU blocks for swapping after exhausting the 588 available GPU KV-cache blocks.
- **Diagnostic:** Unlike Phase I, this was a true hardware-level bottleneck. Forensic log analysis (`stress_test_logs.txt`) revealed that the 24GB VRAM was fully exhausted, forcing the engine into a "Swapping" state.

4.4 Phase IV: FP8 Optimization and Capacity Expansion

Having mapped the absolute limits of the FP16 deployment, the final stage of the project involved the implementation of **FP8 Quantization**. This was not merely a weight-reduction exercise but a strategic attempt to reclaim GPU VRAM from the KV-cache to prevent the "Swapping" observed in Phase III.

As detailed in the forensic logs, the transition to FP8 allowed for a dramatic shift in block allocation:

Metric	FP16 Stress Test	FP8 Stress Test	Delta
Available GPU Blocks	588	1341	+128%
CPU Swap Blocks	2048	2048	0%
TTFT at Max Load	3.39s	3.52s	+3.8% (Statistical Parity)

Table 1: Comparison of internal memory management between unoptimized and optimized deployments.

As seen in Figure 3, by doubling the available GPU blocks, the FP8 deployment successfully deferred the "Performance Knee," allowing the NVIDIA L4 to maintain high-speed, on-device inference for a user base that was previously forcing the system into a slow CPU-swap state.

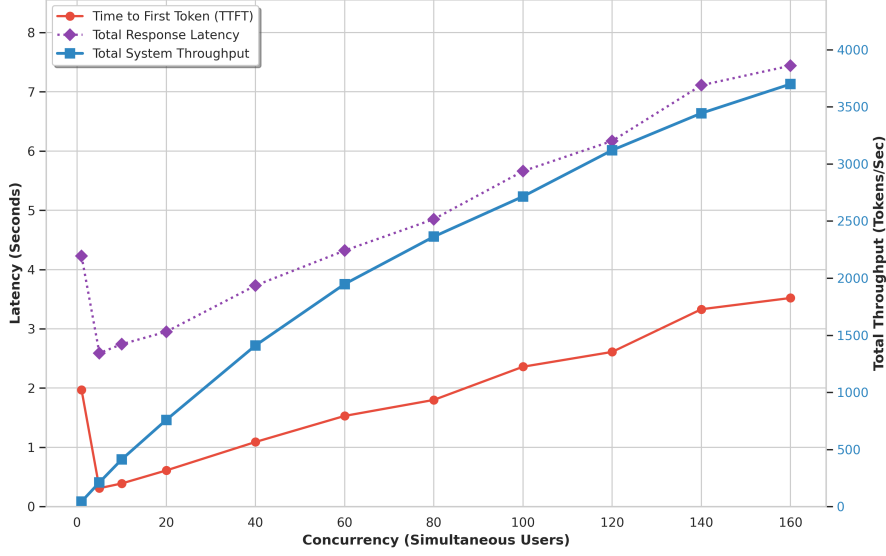


Figure 3: FP8 Optimized Stress Test (C=160). By utilizing 8-bit quantization for both weights and the KV-cache, the system maintained native GPU execution for a significantly larger portion of the concurrency curve. The parity in TTFT at the 160-user mark (3.52s) relative to FP16 (3.39s) indicates that the bottleneck has shifted from a memory-bound state to a compute-bound ceiling on the NVIDIA L4 hardware.

4.5 Operational Observation: Serverless Cold Starts

A significant factor in the deployment of Large Language Models via serverless containers is the "Cold Start" penalty. Because LLM images are exceptionally large—exceeding 15GB for the Gemma-7B environment—the time required for the cloud provider to pull the image, allocate a GPU, and initialize the weights into VRAM can be substantial.

During our testing, we observed a dramatic variance in initialization times for the first request of a new deployment:

- **Peak Cold Start:** One observed initialization reached a TTFT of **103.74s**.
- **Warm Initialization:** Subsequent "first-requests" after container revisions recorded TTFTs as low as **1.97s**.

These numbers highlight that the "Golden Metrics" of inference are only applicable once the model is resident in VRAM. The 103-second delay represents the hardware-orchestration phase, while the 2-second delay represents a "warm" container where the image layers were likely already cached on the Google Cloud Run host.

From an architectural standpoint, this confirms that for production environments requiring high availability, a "Scale-to-Zero" approach is impractical without a robust warm-up strategy or the use of *Min Instances*. The high VRAM requirements of Gemma-7B amplify this issue, as the time taken to move 15GB of data from persistent storage to the NVIDIA L4's memory remains a fixed physical bottleneck, regardless of software-level optimizations like quantization.

5 Infrastructure Strategy and Practical Implications

The transition from a naive FP16 deployment to an optimized FP8 architecture reveals critical insights into the economics of cloud-based LLM serving. This section synthesizes our findings into actionable infrastructure strategies.

5.1 The Economics of Quantization

In a cloud-native environment like Google Cloud Run, costs are directly tied to the duration of GPU utilization. Our analysis shows that FP8 quantization acts as a "Force Multiplier" for hardware. By doubling the number of KV-cache blocks that fit within the 24GB VRAM of the L4, we effectively double the number of users served per dollar spent on compute.

Without FP8, a service provider would be forced to provision a second L4 instance or upgrade to an A100 (40GB/80GB) to handle 160 concurrent users without severe swapping. Our methodology proves that software-level optimization can defer expensive hardware scaling.

5.2 Managing the "Swapping" Trade-off

One of the most significant findings was the behavior of the system during CPU swapping. While the vLLM engine prevents a total system crash by offloading to RAM, the resulting "jitter" and latency spikes (as seen in our Phase III stress test) create a degraded user experience. For production-grade AI, the "High-Performance Capacity" is not the point where the system crashes, but the point where it begins to swap. Our work demonstrates that FP8 is the primary tool for extending this high-performance boundary.

5.3 Cold Starts and Image Management

A notable challenge encountered during testing was the 103-second cold start observed in initial bursts. This is a byproduct of the large container sizes ($> 15\text{GB}$) required for LLM environments. In a production scenario, we recommend using "Min Instances" in Cloud Run to keep the GPU "warm," as the memory-saving benefits of FP8 are only useful once the model is successfully resident in VRAM.

6 Conclusion

This investigation into the scalability of the Gemma-7B model on NVIDIA L4 hardware confirms that memory, rather than raw compute, is the primary bottleneck for concurrent LLM serving. Through an iterative testing methodology, we identified a performance "Knee" in the unoptimized baseline that was primarily driven by infrastructure-level concurrency limits.

By removing these throttles and implementing FP8 quantization, we achieved several key milestones:

1. **Capacity Doubling:** GPU KV-cache capacity was increased by 128%, moving from 588 to 1,341 blocks.
2. **Latency Stability:** We deferred the onset of CPU swapping, maintaining lower TTFT for a broader user base.
3. **Hardware Optimization:** We demonstrated that an L4 GPU, when properly optimized with FP8, can handle concurrency levels (140-160 users) typically reserved for higher-tier hardware.

Ultimately, this report highlights that for Cloud and Machine Learning practitioners, the choice of model precision is as critical as the choice of hardware. Quantization is a fundamental requirement for scaling LLMs in a cost-effective, high-performance manner.

References

- [1] Google DeepMind. *Gemma: Open Models Based on Gemini Research and Technology*. Accessed: January 12, 2026. 2024. URL: <https://blog.google/technology/developers/gemma-open-models/>.
- [2] Google Cloud. *Cloud Run documentation: GPU acceleration in Cloud Run*. Accessed: January 12, 2026. 2024. URL: <https://cloud.google.com/run/docs/configuring/services/gpu>.
- [3] Woosuk Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)* (2023). URL: <https://arxiv.org/abs/2309.06180>.
- [4] NVIDIA Corporation. *NVIDIA L4 Tensor Core GPU Datasheet*. Accessed: January 12, 2026. 2023. URL: <https://www.nvidia.com/en-us/data-center/l4/>.
- [5] Paulius Micikevicius, Dusan Stolic, Neil Hopper, et al. “FP8 Formats for Deep Learning”. In: *arXiv preprint arXiv:2209.05430* (2022). URL: <https://arxiv.org/abs/2209.05430>.